

In **Datasets**, the data is internally stored in the **Apache Arrow format**.
Apache Arrow is **fast and memory-efficient**, which makes it suitable for large datasets.

However, you can still **view and work with the same data as a Pandas DataFrame** if needed.
After finishing your analysis in Pandas, you can also **convert the DataFrame back into a Dataset**.

1) What does `set_format("pandas")` do?

```
drug_dataset.set_format("pandas")
```

This means:

- The **actual storage format of the data does not change**.
- Only the **output format changes** when you retrieve data from the dataset.

So:

- Internally → still **Apache Arrow**
- Output when accessed → **Pandas DataFrame**

2) Why does `drug_dataset["train"]` still remain a Dataset?

This is an important point.

```
drug_dataset["train"]
```

This still returns a **Dataset object**.

The reason is that `set_format()` **does not convert the dataset into a DataFrame**. Instead, it only changes the **output format when accessing items** from the dataset.

Technically, it modifies the `__getitem__()` method of the dataset.

3) How do we get a DataFrame then?

```
drug_dataset["train"][:3]
```

Here `[:3]` is a **slice operation**.

Since the output format is now set to "**pandas**", slicing the dataset returns a **pandas.DataFrame**.

Example:

```
drug_dataset["train"][:3]
```

The output will be a **pandas.DataFrame** containing **3 rows**.

4) Converting the whole training set into a DataFrame

```
train_df = drug_dataset["train"][:]
```

Here `[:]` means **select all rows**.

Therefore `train_df` becomes a **full pandas DataFrame** containing the entire training set.

5) Now we can use all Pandas functionality

Since `train_df` is now a Pandas DataFrame, we can use any Pandas operation.

For example, computing the **frequency of each condition**:

```
frequencies = (  
    train_df["condition"]  
    .value_counts()  
    .to_frame()  
    .reset_index()  
    .rename(columns={"index": "condition", "count": "frequency"})  
)
```

Step-by-step explanation:

```
train_df["condition"]
```

Selects only the `condition` column.

```
.value_counts()
```

Counts how many times each condition appears.

```
.to_frame()
```

Converts the result from a Series into a DataFrame.

```
.reset_index()
```

Converts the index into a normal column.

```
.rename(...)
```

Renames the column names.

Example result:

condition	frequency
birth control	27655
depression	8023

6) Converting a DataFrame back to a Dataset

```
from datasets import Dataset
```

```
freq_dataset = Dataset.from_pandas(frequencies)
```

Here `frequencies` is a **Pandas DataFrame**.

`Dataset.from_pandas()` converts it into a **Hugging Face Dataset**.

So the flow becomes:

Dataset → DataFrame → Dataset

7) Why do we need `reset_format()`?

```
drug_dataset.reset_format()
```

This restores the **default output format** of the dataset.

That means:

- It will **no longer return Pandas DataFrames**
- It will return the **normal dataset-style output**

This is useful when you finish your Pandas analysis and want to continue using dataset operations.

8) Ways to save a Dataset

The library automatically caches datasets, but you may want to **save them permanently to disk**.

Three common formats are supported:

Format	Function
Arrow	<code>save_to_disk()</code>
CSV	<code>to_csv()</code>
JSON	<code>to_json()</code>

9) Saving in Arrow format

```
drug_dataset_clean.save_to_disk("drug-reviews")
```

This saves the dataset as a **folder**.

`drug_dataset_clean` is likely a **DatasetDict** that contains multiple splits:

- train
- validation

- test

Each split will have its own **dataset.arrow** file.

Why Arrow is useful:

- Fast
- Columnar format
- Efficient for large datasets

10) Loading the saved dataset again

```
from datasets import load_from_disk
```

```
drug_dataset_reloaded = load_from_disk("drug-reviews")
```

This loads the dataset back from disk.

The result will again be a **DatasetDict**.

11) Saving as JSON or CSV requires separate files

When using JSON or CSV, each split must be saved separately.

That is why a loop is used:

```
for split, dataset in drug_dataset_clean.items():  
    dataset.to_json(f"drug-reviews-{split}.jsonl")
```

This creates:

```
train → drug-reviews-train.jsonl  
validation → drug-reviews-validation.jsonl  
test → drug-reviews-test.jsonl
```

12) What is **jsonl**?

`jsonl` means **JSON Lines**.

This means:

- Each line contains **one JSON object**
- One line represents **one row of the dataset**

Example:

```
{"patient_id":141780,"drugName":"Escitalopram", ...}
```

This format is useful for handling **large datasets efficiently**.

13) Loading JSON files again

```
data_files = {  
    "train": "drug-reviews-train.jsonl",  
    "validation": "drug-reviews-validation.jsonl",  
    "test": "drug-reviews-test.jsonl",  
}
```

```
drug_dataset_reloaded = load_dataset("json", data_files=data_files)
```

Here we tell `load_dataset()` which file belongs to which split.

The result will again be a **DatasetDict**.

Most Important Points

```
set_format("pandas")
```

Changes only the **output format**, not the data storage.

```
drug_dataset["train"]
```

Still returns a **Dataset**.

```
drug_dataset["train"][:]
```

Returns a **DataFrame** because we slice the dataset.

```
Dataset.from_pandas(df)
```

Converts a **DataFrame** into a **Dataset**.

```
reset_format()
```

Restores the default output format.

```
save_to_disk()
```

Saves the dataset in **Arrow format**.

```
load_from_disk()
```

Loads the saved Arrow dataset.

Very Small Example

```
from datasets import Dataset
```

```
import pandas as pd
```

```
data = {  
    "name": ["A", "B", "C"],  
    "score": [80, 90, 85]  
}
```

```
ds = Dataset.from_dict(data)
```

```
ds.set_format("pandas")
```

```
df = ds[:] # now this becomes a DataFrame  
print(type(df))
```

```
new_ds = Dataset.from_pandas(df)  
print(new_ds)
```

This demonstrates the workflow:

Dataset → Pandas DataFrame → Dataset again.